

**РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ
НАРОДОВ**

Факультет физико-математических и естественных наук

**Кафедра прикладной информатики и теории
вероятностей**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7.**

Дисциплина: Архитектура компьютера

Студент: Павлушина Виктория Александровна

Группа: НКАбд-05-25

МОСКВА 2025 г.

Оглавление

1. Цель работы:	3
2.Порядок выполнения лабораторной работы	4
2.1. Реализация переходов в NASM	4
2.2. Изучение структуры файлы листинга	10
2.3. Задание для самостоятельной работы.....	13
3. Вывод:	17

1. Цель работы:

Изучение команд условного и безусловного переходов.

Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

2. Порядок выполнения лабораторной работы

2.1. Реализация переходов в NASM

Создадим каталог для программ лабораторной работы № 7, перейдём в него и создадим файл *lab7-1.asm*:

```
vapavlushina@dk7n03 ~ $ mkdir ~/work/arch-pc/lab07
vapavlushina@dk7n03 ~ $ cd ~/work/arch-pc/lab07
vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ touch lab7-1.asm
```

Рисунок 1 Создадим папку lab07, перейдем в нее и создадим файл lab7-1.asm

Инструкция `jmp` в NASM используется для реализации безусловных переходов. Рассмотрим пример программы с использованием инструкции `jmp`. Введём в файл *lab7-1.asm* текст программы из листинга 7.1.

```
%include    'in_out.asm'           ; подключение внешнего файла

SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0

SECTION .text
GLOBAL _start
_start:

    jmp _label2

_label1:
    mov eax, msg1    ; Вывод на экран строки
    call sprintf     ; 'Сообщение № 1'

_label2:
    mov eax, msg2    ; Вывод на экран строки
    call sprintf     ; 'Сообщение № 2'
```

```

_label3:
    mov  eax, msg3    ; Вывод на экран строки
    call sprintf      ; 'Сообщение № 3'

_end:
    call quit         ; вызов подпрограммы завершения

```

Рисунок 2 Листинг 7.1.

Создадим исполняемый файл и запустим его:

```

vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ nasm -f elf lab7-1.asm
vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ ld -m elf_i386 -o lab7-1 lab7-1.o
vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ ./lab7-1
Сообщение № 2
Сообщение № 3

```

Рисунок 3 Запустили файл lab7-1

Инструкция `jmp` позволяет осуществлять переходы не только вперед но и назад. Изменим программу таким образом, чтобы она выводила сначала ‘Сообщение № 2’, потом ‘Сообщение № 1’ и завершала работу. Для этого в текст программы после вывода сообщения № 2 добавим инструкцию `jmp` с меткой `_label1` (т.е. переход к инструкциям вывода сообщения № 1) и после вывода сообщения № 1 добавим инструкцию `jmp` с меткой `_end` (т.е. переход к инструкции `call quit`).

Создадим файл *lab7-01.asm* и введём в него текст программы из листинга 7.2. :

```

vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ touch lab7-01.asm

```

Рисунок 4 Создали файл lab7-01.asm

```

%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
jmp _label1
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
_end:
call quit ; вызов подпрограммы завершения

```

Рисунок 5 текст листинга 7.2.

Теперь запустим файл.

```

vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ nasm -f elf lab7-01.asm
vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ ld -m elf_i386 -o lab7-01 lab7-01.o
vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ ./lab7-01
Сообщение № 2
Сообщение № 1
vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ █

```

Рисунок 6 Запустили файл lab7-01

Изменим текст программы изменив инструкции `jmp`, чтобы вывод программы был следующим:

```
user@dk4n31:~$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1
user@dk4n31:~$
```

Мы создадим копию файла *lab7-02.asm* и произведём изменения в нём.

Оттранслируем файл и запустим его.

```
vapavlushina@dk7n07 ~/work/arch-pc/lab07 $ nasm -f elf lab7-02.asm
vapavlushina@dk7n07 ~/work/arch-pc/lab07 $ ld -m elf_i386 -o lab7-02 lab7-02.o
vapavlushina@dk7n07 ~/work/arch-pc/lab07 $ ./lab7-02
Сообщение № 3
Сообщение № 2
Сообщение № 1
vapavlushina@dk7n07 ~/work/arch-pc/lab07 $ █
```

Рисунок 7 Корректно выполнили задание

```
%include 'in_out.asm' ; подключение внешнего файла

SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0

SECTION .text
GLOBAL _start
_start:
    jmp _label3      ; Первый переход к выводу сообщения №3

_label1:
    mov eax, msg1    ; Вывод на экран строки
    call sprintf     ; 'Сообщение № 1'
    jmp _end

_label2:
    mov eax, msg2    ; Вывод на экран строки
    call sprintf     ; 'Сообщение № 2'
    jmp _label1      ; Переход к выводу сообщения №1

_label3:
    mov eax, msg3    ; Вывод на экран строки
    call sprintf     ; 'Сообщение № 3'
    jmp _label2      ; Переход к выводу сообщения №2

_end:
    call quit        ; вызов подпрограммы завершения
```

Рисунок 8 Текст программы для файла *lab7-02.asm*

Использование инструкции `jmp` приводит к переходу в любом случае. Однако, часто при написании программ необходимо использовать условные переходы, т.е. переход должен происходить если выполнено какое-либо условие. В качестве примера рассмотрим программу, которая определяет и выводит на экран наибольшую из 3 целочисленных переменных: А, В и С. Значения для А и С задаются в программе, значение В вводится с клавиатуры.

Создадим файл *lab7-2.asm* в каталоге *~/work/arch-pc/lab07*. Внимательно изучим текст программы из листинга 7.3 и введём в *lab7-2.asm*:

```
vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ touch lab7-2.asm
```

Рисунок 9 Создание файла *lab7-2.asm*

```
vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ nasm -f elf lab7-2.asm
vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ ld -m elf_i386 -o lab7-2 lab7-2.o
vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ ./lab7-2
Введите В: 1
Наибольшее число: 50
vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ ./lab7-2
Введите В: 2
Наибольшее число: 50
vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ ./lab7-2
Введите В: 750
Наибольшее число: 750
```

Рисунок 10 Запустили файл *lab7-2* и проверили корректность выполнения программы, введя несколько значений


```

#include 'in_out.asm'
section .data
msg1 db 'Введите B: ',0h
msg2 db "Наибольшее число: ",0h
A dd '20'
C dd '50'
section .bss
max resb 10
B resb 10
section .text
global _start
_start:
; ----- Вывод сообщения 'Введите B: '
mov eax,msg1
call sprint
; ----- Ввод 'B'
mov ecx,B
mov edx,10
call sread
; ----- Преобразование 'B' из символа в число
mov eax,B
call atoi ; Вызов подпрограммы перевода символа в число
mov [B],eax ; запись преобразованного числа в 'B'
; ----- Записываем 'A' в переменную 'max'
mov ecx,[A] ; 'ecx = A'
mov [max],ecx ; 'max = A'
; ----- Сравниваем 'A' и 'C' (как символы)
cmp ecx,[C] ; Сравниваем 'A' и 'C'
jg check_B ; если 'A>C', то переход на метку 'check_B',

mov ecx,[C] ; иначе 'ecx = C'
mov [max],ecx ; 'max = C'
; ----- Преобразование 'max(A,C)' из символа в число
check_B:
mov eax,max
call atoi ; Вызов подпрограммы перевода символа в число
mov [max],eax ; запись преобразованного числа в 'max'
; ----- Сравниваем 'max(A,C)' и 'B' (как числа)
mov ecx,[max]
cmp ecx,[B] ; Сравниваем 'max(A,C)' и 'B'
jg fin ; если 'max(A,C)>B', то переход на 'fin',
mov ecx ; ОШИБКА: удален второй операнд [B]
mov [max],ecx
; ----- Вывод результата
fin:
mov eax, msg2
call sprint ; Вывод сообщения 'Наибольшее число: '
mov eax,[max]
call iprintLF ; Вывод 'max(A,B,C)'
call quit ; Выход

```

Рисунок 11 Текст листинга 7.3.

2.2. Изучение структуры файлы листинга

Обычно **nasm** создаёт в результате ассемблирования только объектный файл. Получить файл листинга можно, указав ключ **-l** и задав имя файла листинга в командной строке. Создадим файл листинга для программы из файла *lab7-2.asm*

```
vapavlushina@dk7n03 ~/work/arch-pc/lab07 $ nasm -f elf -l lab7-2.lst lab7-2.asm
```

Рисунок 12 Сделали файл листинга для программы из файла *lab7-2.asm*

Откроем файл листинга *lab7-2.lst*.. Подробно объясним содержимое трёх строк файла листинга:

143	000000BB 80EB30	<1>	sub	bl, 48
144	000000BE 01D8	<1>	add	eax, ebx
145	000000C0 BB0A000000	<1>	mov	ebx, 10

Рисунок 13 Строки, которые мы будем объяснять

Строка 143: **sub bl, 48**

- **Операция:** Вычитание
- **Что делает:** Преобразует символ цифры в соответствующее числовое значение.
 - В кодировке ASCII символ '0' имеет код 48 (0x30)
 - Вычитая 48 из значения в регистре BL (младший байт EBX), мы преобразуем символ цифры в фактическое число (0-9)
 - Например: символ '5' (код 53) $\rightarrow 53 - 48 = 5$

Строка 144: `add eax, ebx`

- **Операция:** Сложение
- **Что делает:** Добавляет преобразованную цифру к накапливаемому результату.

Регистр EAX обычно содержит текущее накопленное число. К нему добавляется значение из EBX (которое теперь содержит цифру 0-9 после преобразования).

Строка 145: `mov ebx, 10`

- **Операция:** Перемещение (присваивание)
- **Что делает:** Устанавливает значение 10 в регистр EBX. Подготавливает регистр EBX для следующей операции.

Откроем файл с программой *lab7-2.asm* и в инструкции с двумя операндами удалим один операнд.

```
16 ; ----- Ввод 'B'
17 mov ecx, B
18 mov edx, 10
19 call sread
20 ; ----- Преобразование 'B' из символа в число
21 mov eax, B
22 call atoi ; Вызов подпрограммы перевода символа в число
23 mov [B], eax ; запись преобразованного числа в 'B'
24 ; ----- Записываем 'A' в переменную 'max'
25 mov ecx, [A] ; 'ecx = A'
26 mov [max], ecx ; 'max = A'
27 ; ----- Сравниваем 'A' и 'C' (как символы)
28 cmp ecx, [C] ; Сравниваем 'A' и 'C'
29 jg check_B ; если 'A > C', то переход на метку 'check_B',
```

Рисунок 14 Удалим операнд B из 17 строки листинга

Если выполнить трансляцию после удаления операнда, то листинг создастся лишь частично (до строки с ошибкой), а объектный файл не создастся вовсе.

```
vapavlushina@dk7n07 ~/work/arch-pc/lab07 $ nasm -f elf lab7-2.asm
lab7-2.asm:17: error: invalid combination of opcode and operands
lab7-2.asm:25: error: invalid combination of opcode and operands
vapavlushina@dk7n07 ~/work/arch-pc/lab07 $ █
```

Рисунок 15 Вылезла ошибка

2.3. Задание для самостоятельной работы

1. Напишем программу нахождения наименьшей из 3 целочисленных переменных a, b и c . Значения переменных выберем из табл. 7.5 в соответствии с вариантом, полученным при выполнении лабораторной работы № 6 (У нас это вариант 16). Создадим файл *lab7-3.asm*, впишем в него наш код, преобразуем его в исполняемый файл и проверим корректность выполнения программы.

Таблица 7.5. Значения a, b, c для задания №1.

Номер варианта	Значения a, b, c	Номер варианта	Значения a, b, c
1	17,23,45	11	21,28,34
2	82,59,61	12	99,29,26
3	94,5,58	13	84,32,77
4	8,88,68	14	81,22,72
5	54,62,87	15	32,6,54
6	79,83,41	16	44,74,17
7	45,67,15	17	26,12,68
8	52,33,40	18	83,73,30
9	24,98,15	19	46,32,74
10	41,62,35	20	95,2,61

Рисунок 16 Таблица 7.5.

```
vapavlushina@dk5n14 ~/work/arch-pc/lab07 $ touch lab7-3.asm
```

Рисунок 17 Создали файл *lab7-3.asm*

```
vapavlushina@dk5n14 ~/work/arch-pc/lab07 $ nasm -f elf lab7-3.asm
vapavlushina@dk5n14 ~/work/arch-pc/lab07 $ ld -m elf_i386 -o lab7-3 lab7-3.o
vapavlushina@dk5n14 ~/work/arch-pc/lab07 $ ./lab7-3
Введите a: 44
Введите b: 74
Введите c: 17
Наименьшее число: 17
```

Рисунок 18 Проверяем работу нашей программы

```

#include 'in_out.asm'

SECTION .data
    msg_a db "Введите a: ",0
    msg_b db "Введите b: ",0
    msg_c db "Введите c: ",0
    msg_result db "Наименьшее число: ",0

SECTION .bss
    a resb 10
    b resb 10
    c resb 10
    min resb 10

SECTION .text
global _start

_start:
    ; Ввод переменной a
    mov eax, msg_a
    call sprint
    mov ecx, a
    mov edx, 10
    call sread
    mov eax, a
    call atoi
    mov [a], eax

    ; Ввод переменной b
    mov eax, msg_b
    call sprint
    mov ecx, b
    mov edx, 10
    call sread
    mov eax, b
    call atoi
    mov [b], eax

    ; Ввод переменной c
    mov eax, msg_c
    call sprint
    mov ecx, c
    mov edx, 10
    call sread
    mov eax, c
    call atoi
    mov [c], eax

    ; Находим наименьшее число
    mov eax, [a] ; предполагаем, что a - наименьшее

    ; Сравниваем c b
    cmp eax, [b]
    jnl check_c ; если a < b, проверяем c
    mov eax, [b] ; иначе min = b

check_c:
    ; Сравниваем c c
    cmp eax, [c]
    jnl print_min ; если текущий min < c, выводим результат
    mov eax, [c] ; иначе min = c

print_min:
    mov [min], eax

    ; Вывод результата
    mov eax, msg_result
    call sprint
    mov eax, [min]
    call iprintLF

    call quit

```

Рисунок 19 Текст программы lab7-3.asm

2. Напишем программу, которая для введенных с клавиатуры значений x и a вычисляет значение заданной функции $f(x)$ и выводит результат вычислений. Вид функции $f(x)$ выберем из таблицы 7.6 в соответствии с вариантом, полученным при выполнении лабораторной работы № 6 (У нас это вариант 16). Создадим файл *lab7-4.asm*, преобразуем его в исполняемый файл и проверим корректность выполнения программы для значений x и a из таблицы 7.6.

$$16 \quad \begin{cases} x + 4, & x < 4 \\ ax, & x \geq 4 \end{cases} \quad (1;1)$$

Рисунок 20 Вариант 16 из таблицы 7.6

```
varpavlushina@dk5n14 ~/work/arch-pc/lab07 $ touch lab7-4.asm
```

Рисунок 21 Создали файл lab7-4.asm

```
varpavlushina@dk5n14 ~/work/arch-pc/lab07 $ nasm -f elf lab7-4.asm
varpavlushina@dk5n14 ~/work/arch-pc/lab07 $ ld -m elf_i386 -o lab7-4 lab7-4.o
varpavlushina@dk5n14 ~/work/arch-pc/lab07 $ ./lab7-4
Введите x: 1
Введите a: 1
f(x) = 5
varpavlushina@dk5n14 ~/work/arch-pc/lab07 $ ./lab7-4
Введите x: 7
Введите a: 1
f(x) = 7
```

Рисунок 22 Выполнение программы lab7-4.asm

```

#include 'in_out.asm'

SECTION .data
    msg_x db "Введите x: ",0
    msg_a db "Введите a: ",0
    msg_result db "f(x) = ",0
    newline db 0xA,0

SECTION .bss
    x resb 10
    a resb 10
    result resb 10

SECTION .text
global _start

_start:
    ; Ввод переменной x
    mov eax, msg_x
    call sprint
    mov ecx, x
    mov edx, 10
    call sread
    mov eax, x
    call atoi
    mov [x], eax

    ; Ввод переменной a
    mov eax, msg_a
    call sprint
    mov ecx, a
    mov edx, 10
    call sread
    mov eax, a
    call atoi
    mov [a], eax

    ; Вычисление функции f(x)
    mov eax, [x]      ; загружаем x в eax
    cmp eax, 4        ; сравниваем x с 4

    jnl case_x_less    ; если x < 4, переходим к первому случаю

    ; Случай 2: x ≥ 4, f(x) = a*x
case_x_ge:
    mov eax, [a]      ; загружаем a
    mov ebx, [x]      ; загружаем x
    imul eax, ebx     ; eax = a * x
    jmp print_result

    ; Случай 1: x < 4, f(x) = x + 4
case_x_less:
    mov eax, [x]      ; загружаем x
    add eax, 4        ; eax = x + 4

print_result:
    ; Сохраняем результат
    mov [result], eax

    ; Вывод результата
    mov eax, msg_result
    call sprint
    mov eax, [result]
    call iprintLF

    ; Завершение программы
    call quit

```

Рисунок 23 Листинг программы lab7-4.asm

3. Вывод:

Изучила команды условного и безусловного переходов.
Приобрела навыки написания программ с использованием переходов.
Ознакомилась с назначением и структурой файла листинга.